Exp06 Report — Decoding Golden Cove: A Bytes-to-uOps Simulation

Computer Organization Lab

Deetya Ashish Mehta (Roll Number: CS23I1032)

September 2026

Contents

Summary 1

1. What the Simulator Models 1

2. Encoding Verification 3

3. Workload Results 3

4. My Own Investigation 6

5. Answering the Final Question 7

Appendix – Reproducing These Results 8

Summary

This report presents a Python-based functional model of Intel Golden Cove's legacy instruction decode path: a 32-byte-per-cycle fetch window feeding six parallel decoder lanes (D0 through D5), which turn raw x86-64 bytes into an abstract uop stream. I implement the six required stages explicitly rather than as one opaque function – boundary/length detection, prefix and REX handling, opcode lookup, ModRM/SIB/ displacement/immediate decoding, instruction classification, and finally uop generation – and drive the model with the five workloads specified in the lab (W1-W5) plus a sixth stream I designed myself to probe a specific hypothesis about instruction fusion. All conclusions below are backed by the raw decoder traces, per-cycle/per-instruction CSVs, and plots produced by `src/run_all.py`, all included under `results/`.

1. What the Simulator Models

1.1 Decode as a pipeline, not a function call

```
bytes --[length_decode.decode_one_instruction]--> prefixes/REX --> opcode
      --[ModRM/SIB/disp/imm]--> operand fields
```

```
--[uop_rules.classify]--> instruction class
--[simulator: fusion + MSROM resolution]--> uop(s)
```

`length_decode.py` owns everything through operand-field decoding; `uop_rules.py` is purely a classification table (never touches raw bytes); `simulator.py` owns the six-wide, cycle-by-cycle scheduling policy, including the two "special decode path" behaviours (macro-fusion and MSROM) described next.

1.2 Per-cycle admission policy

A cycle accepts instructions into lanes D0, D1, … in order, and stops admitting new instructions as soon as it would either (a) exceed six instructions in the cycle, or (b) exceed 32 bytes consumed in the cycle. Whichever limit triggers first ends that cycle; unfinished stream is carried forward (nothing is dropped or re-fetched).

1.3 The uop model used throughout this lab

Because the exact internal uop counts Intel uses are not public, I fixed an explicit rule set and applied it identically to every workload, so that comparisons between workloads are meaningful:

- simple register-form ALU ops (ADD/SUB/XOR, register-register or register-imm8) → **1 uop**;
- a load or a store to/from memory → **1 uop** each;
- LEA → **1 uop**, regardless of how complex its SIB/displacement addressing is, since it only performs address arithmetic and never touches the data cache;
- a memory read-modify-write ALU op (e.g. `ADD [mem], reg`) → **2 uops** (a fused load+op, plus a separate store) — this is my chosen concrete instance of "uop expansion";
- CMP/TEST in register form → **1 uop**, and marked as eligible for fusion with an immediately following conditional jump;
- a conditional jump (Jcc) → **1 uop** normally, or **0 additional uops** if the decoder fuses it into a preceding CMP/TEST;
- CPUID, RDTSC, ENTER, REP-prefixed string moves, and LOCK'd atomic read-modify-writes are all routed to the **MSROM** path, with fixed costs of 4, 2, 3, 4 (symbolic — the true count is RCX-dependent at runtime, not something decode-time analysis can know), and 5 uops respectively.

1.4 Two "special path" rules that matter for the results below

Fusion is a same-cycle, adjacent-lane phenomenon in this model, not merely a byte-adjacency one: a CMP/TEST decoded into lane k only fuses with a following Jcc if that Jcc is decoded into lane *k+1* of the *same* cycle. If the CMP/TEST happens to occupy D5, there is no D6 to fuse into within that cycle, and the Jcc — even though it is still the very next thing in the byte stream — is deferred to D0 of the next cycle and decoded independently.

MSROM closes its cycle immediately. Once a lane decodes an MSROM-class instruction, no further lane is used in that cycle, no matter how much byte or lane budget is technically left over. This reflects the hand-off from the parallel legacy decoders to the single-issue microcode sequencer.

2. Encoding Verification

Before writing any decode logic, I hand-encoded all workload byte strings against the x86-64 ISA reference and then cross-checked every one of them with the Capstone disassembler in a throwaway script (not part of the simulator itself) to make sure my assumed instruction boundaries, mnemonics and operands were correct. All six streams (W1-W5, plus the custom stream in Section 4) matched Capstone's output byte-for-byte before a single line of `length_decode.py` was written against them.

3. Workload Results

Full decoder traces: `results/traces/<workload>.txt`. Machine-readable data: `results/per_cycle.csv`, `results/per_instruction.csv`, `results/summary.csv`.

Workload	Instr/cycle	Bytes/cycle	uops/cycle	Fusion	MSROM
W1 (dense, simple)	4.0	12.0	4.0	0	0
W2 (variable length)	3.5	16.0	3.5	0	0
W3 (fusion)	4.0	10.0	2.0	4	0
W4 (expansion/MSROM)	1.4	4.4	4.2	0	5
W5 (mixed realistic)	3.0	10.0	4.0	1	1

Per-workload observations:

- **W1** — all 6 lanes fill using only 18/32 bytes; decoder slots run out well before fetch bandwidth does.
- **W2** — even with an 8-byte instruction present, 6 still pack into 27/32 bytes; the lane cap keeps binding first.
- **W3** — 8 instructions collapse to 4 uops, since every CMP/TEST;Jcc pair here falls on adjacent lanes and fuses.
- **W4** — instructions/cycle craters to 1.4, yet uops/cycle (4.2) stays comparable to healthier workloads; MSROM is the real limiter.
- **W5** — a "full" 6-lane cycle (fusion and RMW together) is followed by a single-MSROM-instruction cycle: two bottleneck shapes in one workload.

(A per-question walkthrough of these results follows in Section 3.1.)

Note W1 and W3 sit at the same instructions/cycle bar height (4.0) but very different uops/cycle bar heights — a visual restatement of Section 3.1's Q5 answer.

Every bar here sits well under the dashed 32-byte line; across this lab's workloads, byte bandwidth was never actually the bottleneck — see Section 5 for when it would be.

W3 sits below the 1.0 reference line (fusion shrinks effective uop count); W4 sits well above it (MSROM inflates it).

3.1 Answering the assignment's guiding questions

1. **All six lanes useful?** Only in cycles where at least six short, independently decodable instructions are available with no MSROM instruction among them — true for cycle 0 of W1, W2, W3, and W5, but never true anywhere in W4.

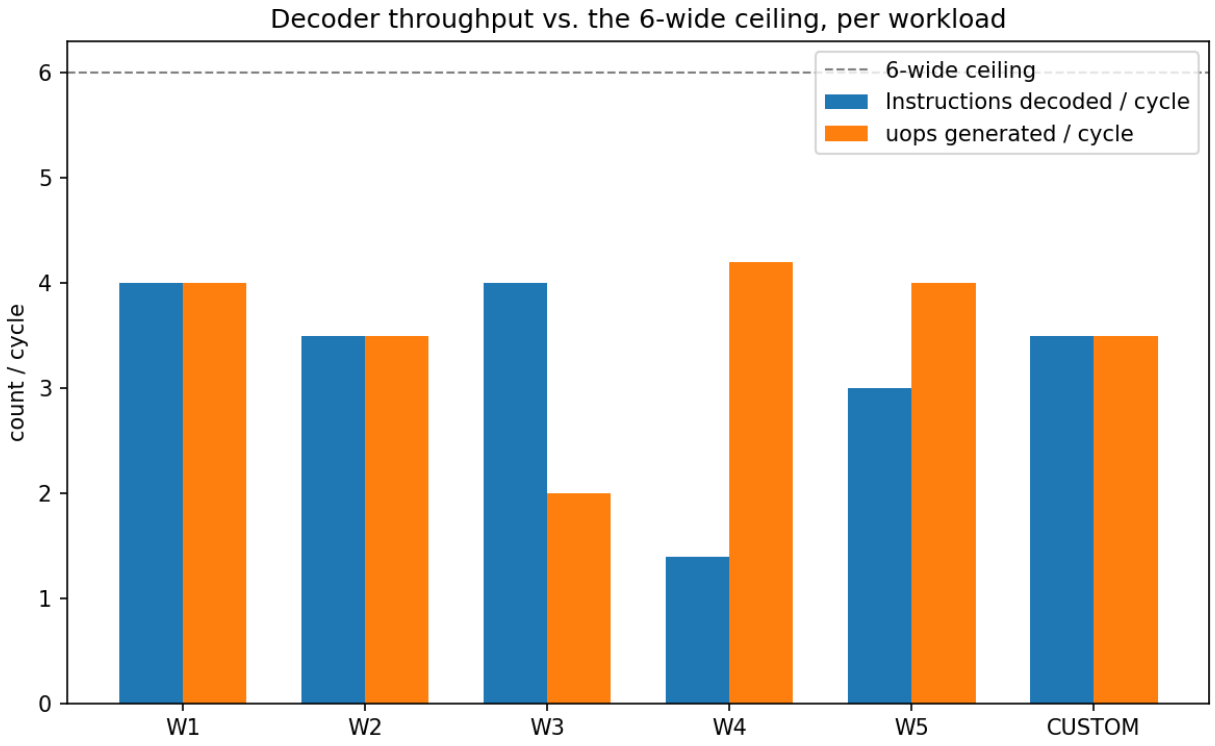


Figure 1: Instructions vs. uops per cycle, six-wide ceiling marked.

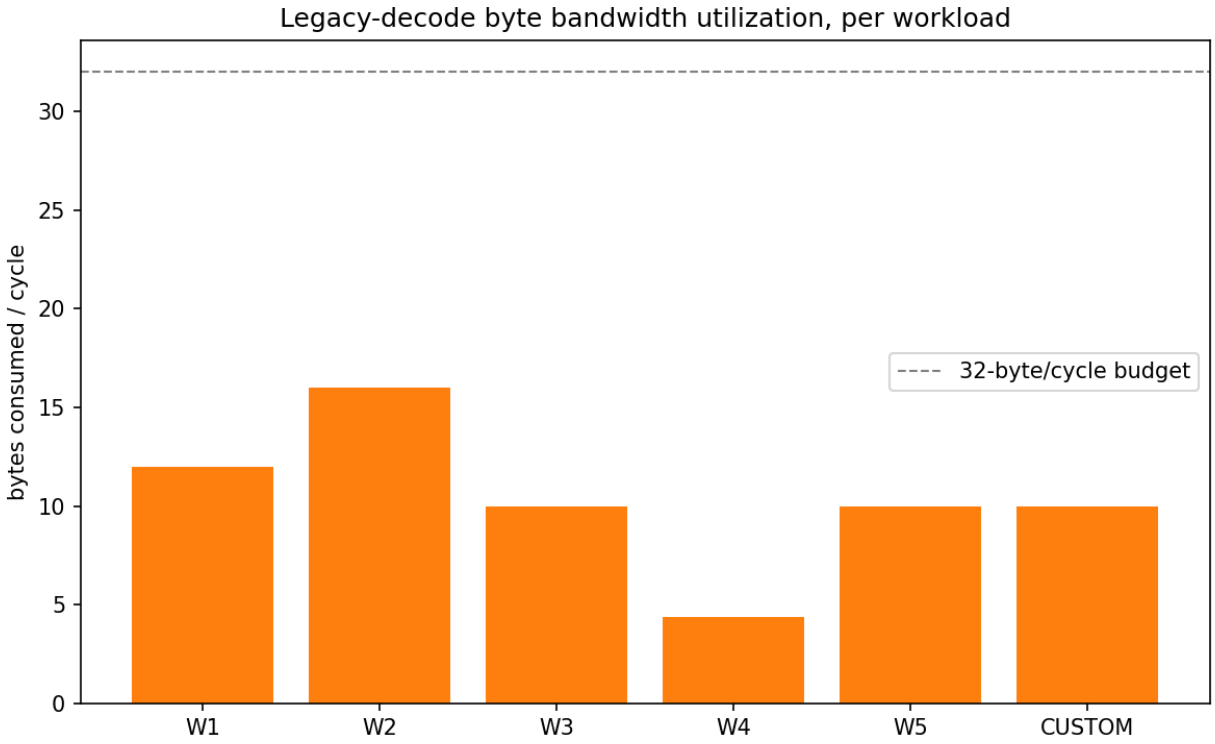


Figure 2: Bytes consumed per cycle against the 32-byte/cycle budget.



Figure 3: uops produced per architectural instruction, by workload.

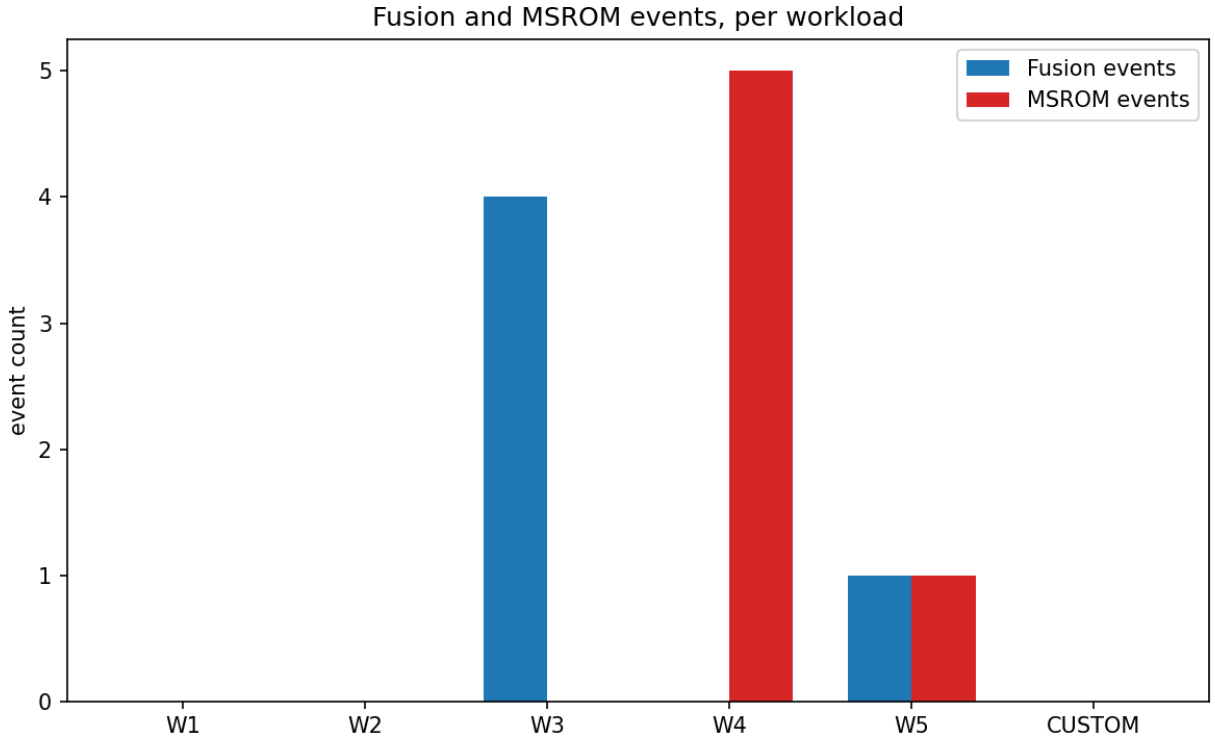


Figure 4: Fusion vs. MSROM event counts, by workload.

2. **Six instructions every cycle for every stream?** No — W4's own data disproves it: 7 instructions needed 5 cycles, because each MSROM instruction forces a single-issue cycle regardless of what else was available to decode.
3. **Effect of very different instruction lengths?** Length variety (W2) changed *how many bytes* were consumed per cycle but did not, by itself, change *how many instructions* fit — the lane cap stayed the binding constraint.
4. **Effect of ModRM/SIB/disp/imm fields?** They lengthen the instruction and describe its operand, but in this model they only change the *uop count* when a memory **write** must be computed on top of a memory **read** (the RMW case) — complex addressing by itself, as LEA shows, adds no uops at all.
5. **Same instruction count, different uop counts?** Yes: W1 and W3 both decode exactly 8 architectural instructions; W1 produces 8 uops (no fusion available), W3 produces only 4 (four fused pairs).
6. **What changes under fusion?** The Jcc is still decoded (occupies a lane, still consumes stream bytes, still counts toward "instructions decoded") but stops contributing its own uop — its work folds into the preceding CMP/TEST's single uop.
7. **What happens on the MSROM path?** The lane reports one instruction decoded, but its uop count jumps sharply (2 to 5 uops, per the table in Section 1.3), and every other lane in that cycle is forced idle even when bytes/lanes were still technically free.
8. **Is instruction count alone enough to understand decoder behaviour?** No. W4's 1.4 instructions/cycle looks like a badly underused decoder until you also look at its 4.2 uops/cycle, which is in the same range as W1's and W5's — the front end is doing comparable *work*, just concentrated in fewer, "heavier" instructions per cycle.
9. **What limited each workload?** W1, W2, W3: the 6-lane cap (byte budget was never close to exhausted in any of them). W4: MSROM residency, unconditionally. W5: the limiting factor actually *changes cycle to cycle within the same workload* — lane-bound in cycle 0, MSROM-bound in cycle 1.

4. My Own Investigation

Hypothesis before running anything: I expected that a CMP followed immediately by a Jcc in the byte stream — with no other bytes in between — would always fuse, since "immediately followed in the instruction stream" is the informal way fusion is usually described. I designed a stream to test whether lane *position* could break that, independent of byte adjacency.

Stream (`src/workloads.py`, `CUSTOM_fusion_across_cycle`):

```
48 01 D8   ADD RAX, RBX     (D0)
48 01 C8   ADD RAX, RCX     (D1)
48 01 F0   ADD RAX, RSI     (D2)
48 01 C1   ADD RCX, RAX     (D3)
48 01 CA   ADD RDX, RCX     (D4)
48 39 D8   CMP RAX, RBX     (D5 -- deliberately the last lane of the cycle)
75 05      JNE +5           (next byte after the CMP, no gap)
```

The five ADD instructions before the CMP were chosen specifically so the CMP would be forced into the sixth and final lane of its cycle.

What the simulator actually produced:

```
Cycle 0 -> D0..D4: ADD x5 (1 uop each); D5: CMP RAX,RBX (1 uop). Fusion events = 0.
Cycle 1 -> D0: JNE +5 (1 uop, independent). Fusion events = 0.
```

Phenomenon: the fusion I expected did not happen — despite this being the exact same `48 39 D8 75 05` byte pair that fused four separate times in W3.

Why: `simulator.py`'s fusion check compares the current instruction only against the *previous lane in the current cycle's lane list*. Since the packing loop's own admission rule (`while len(lanes) < 6`) closes the cycle right after D5 is filled, there simply is no lane for the JNE to occupy in cycle 0 — it starts cycle 1 with an empty "previous lane," so the fusion condition never has anything to compare against. This confirms that, in this model (and consistent with how real front ends are documented to behave), macro-fusion is a *same-decode-group* event: the two instructions must be decoded together in one cycle, not merely be neighbours in program order. A practical implication: whether a given CMP;Jcc pair in real code actually gets the fusion benefit can depend on the lengths and count of instructions immediately preceding it in the same basic block — something that is invisible from the branch and comparison instructions themselves.

5. Answering the Final Question

What actually determines whether Golden Cove's six lanes and 32 bytes/cycle are fully used?

Looking across all six workloads, four things determine it, and they don't all matter equally in every stream:

- **The six-lane ceiling binds before the byte ceiling, for compact code.** In every ordinary (non-MSROM) cycle observed in this lab, 6 instructions were reached using well under 32 bytes (18/32 for W1, 27/32 for W2's longest-instruction cycle). The byte budget would only start to matter for streams whose average instruction length exceeds 32 / 6 ~= 5.33 bytes — none of these workloads' ordinary segments reach that.
- **Fusion recovers uop throughput, but only opportunistically.** W3 shows the mechanism working every time it is offered the chance; the custom stream in Section 4 shows the *identical* instruction pair failing to fuse purely because of where the preceding instructions happened to push it in the decode group. Fusion utilization is therefore a property of the surrounding stream layout, not of the fusable pair alone.
- **MSROM instructions are an unconditional override.** The instant one appears, both the lane and byte budgets become irrelevant for the rest of that cycle — seen directly in W4 (single-issue for 5 of 7 instructions) and in W5's second cycle (a single CPUID, alone, despite bytes and lanes still available).
- **"Instructions/cycle" and "uops/cycle" measure different things, and only the latter reflects actual work delivered downstream.** W4 looks starved on instructions/cycle (1.4) but is not starved on uops/cycle (4.2, comparable to W1's 4.0) — so decoder "utilization," properly understood, has to be read off uop throughput rather than instruction throughput.

Put together: for the kind of compact, register-heavy code represented by W1/W2/W3, the six decoder lanes are the practical bottleneck almost every cycle — the 32-byte/cycle figure is headroom this lab's workloads never came close to using. Fusion partially compensates for the lane limit but is contingent on decode-group alignment rather than guaranteed by program order. And any MSROM-class instruction, however infrequent, dominates whatever cycle it appears in — which is

exactly why W4, built entirely from "special path" instructions, is the true worst case for front-end utilization in this lab, not W2's longer, memory-touching instructions.

Appendix – Reproducing These Results

```
python3 src/run_all.py
```

This single command regenerates every trace under `results/traces/`, every CSV under `results/`, and every plot under `results/plots/` referenced in this report.